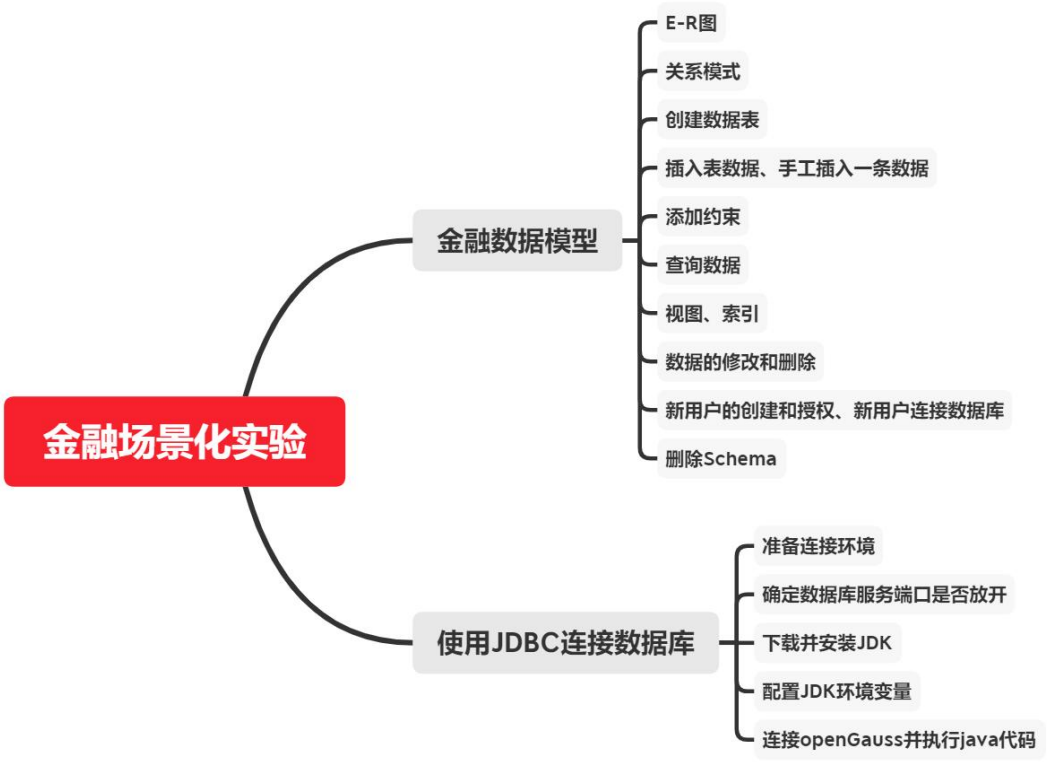
实　　验　　报　　告

学　　号	2108001 2078	姓　　名	姚嫣如	专业班级	网络空间安全 23
课程名称	**数据库系统**			学期	**2025 年秋季学期**
任课教师	刘艳艳	完成日期	2025/12/26	上机课时间	2025/12/24
实 验 名 称	openGauss 场景化综合应用实验				

一、实验要求（10%）

根据实验指导书，以金融行业为场景，设计数据库模型，并使用华为 openGauss 数据库构建金融场景下的数据库。通过对数据库中的对象（表、约束、视图、索引等）创建，掌握基础 SQL 语法，并通过对表中数据对的增删改查，模拟金融场景下的业务实现。

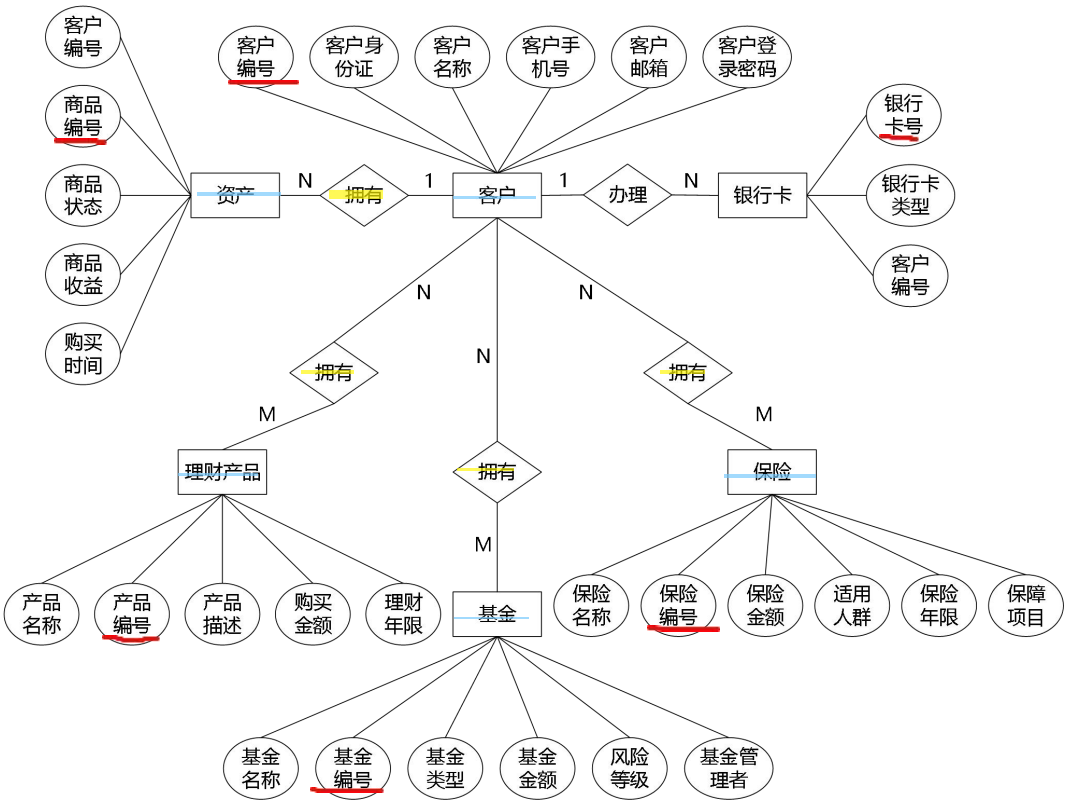


二、实验内容及步骤（80%）

（一）金融数据模型

假设 A 市 C 银行为了方便对银行数据的管理和操作，引入了华为 openGauss 数据库。针对 C 银行的业务，本实验主要将对象分为客户、银行卡、理财产品、保险、基金和资产。因此，针对这些数据库对象，本实验假设 C 银行的金融数据库存在着以下关系：客户可以办理银行卡，同时客户可以购买不同的银行产品，如资产，理财产品，基金和保险。那么，根据 C 银行的对象关系，本实验给出了相应的关系模式和 ER 图，并对其进行较为复杂的数据库操作。

1、E-R 图



2、关系模型

（1）对于 C 银行中的 6 个对象，分别建立属于每个对象的属性集合，具体属性描述如下：

- 客户（客户编号、客户名称、客户邮箱，客户身份证，客户手机号，客户登录密码）
- 银行卡（银行卡号，银行卡类型，所属客户编号）
- 理财产品（产品名称，产品编号，产品描述，购买金额，理财年限）
- 保险（保险名称，保险编号，保险金额，适用人群，保险年限，保障项目）
- 基金（基金名称，基金编号，基金类型，基金金额，风险等级，基金管理者）
- 资产（客户编号，商品编号，商品状态，商品数量，商品收益，购买时间）

（2）上述属性对应的编号为：

- Client(c_id, c_name, c_mail, c_id_card, c_phone, c_password)
- bank_card(b_number, b_type, b_c_id)
- finances_product(p_name, p_id, p_description, p_amount, p_year)
- insurance(i_name, i_id, i_amount, i_person, i_year, i_project)
- fund(f_name, f_id, f_type, f_amount, risk_level, f_manager)
- property(pro_c_id,pro_id,pro_status,pro_quantity,pro_income,pro_purchase_time)

（3）对象之间的关系：

- 一个客户可以办理多张银行卡
- 一个客户可有多笔资产
- 一个客户可以购买多个理财产品，同一类理财产品可由多个客户购买
- 一个客户可以购买多个基金，同一类基金可由多个客户购买
- 一个客户可以购买多个保险，同一类保险可由多个客户购买

3、创建数据表

根据 C 银行的场景描述，本实验分别针对客户(client)，银行卡(bank_card)，理财产品(finances_product)，保险(insurance)，基金(fund)和资产(property)创建相应的表。

（1）创建金融数据库 finance

切换到 omm 用户，以操作系统用户 omm 登录数据库主节点，开启数据库服务，并使用 gsql 工具登陆数据库。

```
[omm@yyr-lab ~]$ gs_om -t start
Starting cluster.
=========================================
=========================================
Successfully started.
[omm@yyr-lab ~]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 1.1.0 build 392c0438) compiled at 2020-12-31 20:08:06 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

postgres=#
```

创建数据库 finance，并连接到数据库。

```
postgres=# CREATE DATABASE finance ENCODING 'UTF8' template = template0;
CREATE DATABASE
postgres=# \connect finance
Non-SSL connection (SSL connection is recommended when requiring high-security)
You are now connected to database "finance" as user "omm".
finance=#
```

创建名为 finance 的 schema，并设置 finance 为当前的 schema，将默认搜索路径设为 finance。

```
finance=# CREATE SCHEMA finance;
CREATE SCHEMA
finance=# SET search_path TO finance;
SET
```

（2）客户信息表的创建

在 SQL 编辑框中输入如下语句，创建客户信息表 client：

```
finance=# DROP TABLE IF EXISTS client;
NOTICE:  table "client" does not exist, skipping
DROP TABLE
finance=# CREATE TABLE client
finance-# (
finance(#         c_id INT PRIMARY KEY,
finance(#         c_name VARCHAR(100) NOT NULL,
finance(#         c_mail CHAR(30) UNIQUE,
finance(#         c_id_card CHAR(20) UNIQUE NOT NULL,
finance(#         c_phone CHAR(20) UNIQUE NOT NULL,
finance(#         c_password CHAR(20) NOT NULL
finance(# );
NOTICE:  CREATE TABLE / PRIMARY KEY will create implicit index "client_pkey" for table "client"
NOTICE:  CREATE TABLE / UNIQUE will create implicit index "client_c_mail_key" for table "client"
NOTICE:  CREATE TABLE / UNIQUE will create implicit index "client_c_id_card_key" for table "client
"
NOTICE:  CREATE TABLE / UNIQUE will create implicit index "client_c_phone_key" for table "client"
CREATE TABLE
```

c_id 为 primary key，c_mail、c_id_card、c_phone 为 unique 约束。

（3）银行卡信息表的创建

```
finance=# DROP TABLE IF EXISTS bank_card;
NOTICE:  table "bank_card" does not exist, skipping
DROP TABLE
finance=# CREATE TABLE bank_card
finance-# (
finance(#         b_number CHAR(30) PRIMARY KEY,
finance(#         b_type CHAR(20),
finance(#         b_c_id INT NOT NULL
finance(# );
NOTICE:  CREATE TABLE / PRIMARY KEY will create implicit index "bank_card_pkey" for table "bank_ca
rd"
CREATE TABLE
```

b_number 为 primary key。

（4）理财产品信息表的创建

```
finance=# DROP TABLE IF EXISTS finances_product;
NOTICE:  table "finances_product" does not exist, skipping
DROP TABLE
finance=# CREATE TABLE finances_product
finance-# (
finance(#         p_name VARCHAR(100) NOT NULL,
finance(#         p_id INT PRIMARY KEY,
finance(#         p_description VARCHAR(4000),
finance(#         p_amount INT,
finance(#         p_year INT
finance(# );
NOTICE:  CREATE TABLE / PRIMARY KEY will create implicit index "finances_product_pkey" for table "
finances_product"
CREATE TABLE
```

p_id 为 primary key。

（5）保险信息表的创建

```
finance=# DROP TABLE IF EXISTS insurance;
WARNING:  Session unused timeout.
FATAL:  terminating connection due to administrator command
could not send data to server: Broken pipe
The connection to the server was lost. Attempting reset: Succeeded.
finance=# CREATE TABLE insurance
finance-# (
finance(#         i_name VARCHAR(100) NOT NULL,
finance(#         i_id INT PRIMARY KEY,
finance(#         i_amount INT,
finance(#         i_person CHAR(20),
finance(#         i_year INT,
finance(#         i_project VARCHAR(200)
finance(# );
NOTICE:  CREATE TABLE / PRIMARY KEY will create implicit index "insurance_pkey" for table "insuran
ce"
CREATE TABLE
```

i_id 为 primary key。

（6）基金信息表的创建

```
finance=# DROP TABLE IF EXISTS fund;
NOTICE:  table "fund" does not exist, skipping
DROP TABLE
finance=# CREATE TABLE fund
finance-# (
finance(#         f_name VARCHAR(100) NOT NULL,
finance(#         f_id INT PRIMARY KEY,
finance(#         f_type CHAR(20),
finance(#         f_amount INT,
finance(#         risk_level CHAR(20) NOT NULL,
finance(#         f_manager INT NOT NULL
finance(# );
NOTICE:  CREATE TABLE / PRIMARY KEY will create implicit index "fund_pkey" for table "fund"
CREATE TABLE
```

f_id 为 primary key。

（7）资产信息表的创建

```
finance=# DROP TABLE IF EXISTS property;
NOTICE:  table "property" does not exist, skipping
DROP TABLE
finance=# CREATE TABLE property
finance-# (
finance(#         pro_c_id INT NOT NULL,
finance(#         pro_id INT PRIMARY KEY,
finance(#         pro_status CHAR(20),
finance(#         pro_quantity INT,
finance(#         pro_income INT,
finance(#         pro_purchase_time DATE
finance(# );
NOTICE:  CREATE TABLE / PRIMARY KEY will create implicit index "property_pkey" for table "property
"
CREATE TABLE
```

pro_id 为 primary key。

（8）显示当前所有表格

```
finance=# \dt
                        List of relations
 Schema  |       Name       | Type  | Owner |             Storage
---------+------------------+-------+-------+---------------------------------
 finance | bank_card        | table | omm   | {orientation=row,compression=no}
 finance | client           | table | omm   | {orientation=row,compression=no}
 finance | finances_product | table | omm   | {orientation=row,compression=no}
 finance | fund             | table | omm   | {orientation=row,compression=no}
 finance | insurance        | table | omm   | {orientation=row,compression=no}
 finance | property         | table | omm   | {orientation=row,compression=no}
(6 rows)
```

4、插入表数据

（1）对 client 表进行数据初始化

```
finance=# INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (1,'张一','zh
angyi@huawei.com','340211993010010001','18815650001','gaussdb_001');
815650026','gaussdb_026');
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (27,'王九','wangjiu@hua
wei.com','340211199301010027',INSERT 0 1
finance=# INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (2,'张二','zh
anger@huawei.com','340211993010010002','18815650002','gaussdb_002');
INSERT 0 1
'18815650027','gaussdb_027');
INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (28,'钱一','qianyi@huaw
ei.com','340211993010010028',finance=# INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_p
assword) VALUES (3,'张三','zhangsan@huawei.com','340211993010010003','18815650003','gaussdb_003');
,'INSERT 0 1
finance=# INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (4,'张四','zh
angsi@huawei.com','340211993010010004','18815650004','gaussdb_004');
,INSERT 0 1
finance=# INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (5,'张五','zh
angwu@huawei.com','340211993010010005','18815650005','gaussdb_005');
INSERT 0 1
finance=# INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (6,'张六','zh
angliu@huawei.com','340211993010010006','18815650006','gaussdb_006');
INSERT 0 1
finance=# INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (7,'张七','zh
angqi@huawei.com','340211993010010007','18815650007','gaussdb_007');
INSERT 0 1
finance=# INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (8,'张八','zh
angba@huawei.com','340211993010010008','18815650008','gaussdb_008');
INSERT 0 1
finance=# INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (9,'张九','zh
angjiu@huawei.com','340211993010010009','18815650009','gaussdb_009');
INSERT 0 1
```

查询插入结果：

一共 30 条数据。

```
finance=# select count(*) from client;
 count
-------
    30
(1 row)
```

（2）对 bank_card 表进行数据初始化

```
finance=# INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000001','信用卡',1);

INSERT 0 1
finance=# INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000002','信用卡',3);

INSERT 0 1
finance=# INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000003','信用卡',5);

INSERT 0 1
finance=# INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000004','信用卡',7);

INSERT 0 1
finance=# INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000005','信用卡',9);

INSERT 0 1
finance=# INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000006','信用卡',10)
;
INSERT 0 1
finance=# INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000007','信用卡',12)
;
INSERT 0 1
finance=# INSERT INTO bank_card(b_number,b_type,b_c_id) VALUES ('6222021302020000008','信用卡',14)
;
INSERT 0 1
```

查询插入结果：

一共 20 条数据。

```
finance=# select count(*) from bank_card;
 count
-------
    20
(1 row)
```

（3）对 finances_product 表进行数据初始化

```
finance=# INSERT INTO finances_product(p_name,p_id,p_description,p_amount,p_year) VALUES ('债券',1
,'以国债、金融债、央行票据、企业债为主要投资方向的银行理财产品。',50000,6);
INSERT 0 1
finance=# INSERT INTO finances_product(p_name,p_id,p_description,p_amount,p_year) VALUES ('信贷资
产',2,'一般指银行作为委托人将通过发行理财产品募集资金委托给信托公司，信托公司作为受托人成立信托计
划，将信托资产购买理财产品发售银行或第三方信贷资产。',50000,6);
INSERT 0 1
finance=# INSERT INTO finances_product(p_name,p_id,p_description,p_amount,p_year) VALUES ('股票',3
,'与股票挂钩的理财产品。目前市场上主要以港股挂钩居多',50000,6);
INSERT 0 1
finance=# INSERT INTO finances_product(p_name,p_id,p_description,p_amount,p_year) VALUES ('大宗商
品',4,'与大宗商品期货挂钩的理财产品。目前市场上主要以挂钩黄金、石油、农产品的理财产品居多。',50000
,6);
INSERT 0 1
```

查询插入结果：

一共 4 条数据。

```
finance=# select count(*) from finances_product;
 count
-------
     4
(1 row)
```

（4）对 insurance 表进行数据初始化

```
finance=# INSERT INTO insurance(i_name,i_id,i_amount,i_person,i_year,i_project) VALUES ('健康保险'
,1,2000,'老人',30,'平安保险');
INSERT 0 1
finance=# INSERT INTO insurance(i_name,i_id,i_amount,i_person,i_year,i_project) VALUES ('人寿保险'
,2,3000,'老人',30,'平安保险');
INSERT 0 1
finance=# INSERT INTO insurance(i_name,i_id,i_amount,i_person,i_year,i_project) VALUES ('意外保险'
,3,5000,'所有人',30,'平安保险');
INSERT 0 1
finance=# INSERT INTO insurance(i_name,i_id,i_amount,i_person,i_year,i_project) VALUES ('医疗保险'
,4,2000,'所有人',30,'平安保险');
INSERT 0 1
finance=# INSERT INTO insurance(i_name,i_id,i_amount,i_person,i_year,i_project) VALUES ('财产损失
保险',5,1500,'中年人',30,'平安保险');
INSERT 0 1
```

查询插入结果如下：

一共 5 条数据。

```
finance=# select count(*) from insurance;
 count
-------
     5
(1 row)
```

（5）对 fund 表进行数据初始化

```
finance=# INSERT INTO fund(f_name,f_id,f_type,f_amount,risk_level,f_manager) VALUES ('股票',1,'股
票型',10000,'高',1);
INSERT 0 1
finance=# INSERT INTO fund(f_name,f_id,f_type,f_amount,risk_level,f_manager) VALUES ('投资',2,'债
券型',10000,'中',2);
INSERT 0 1
finance=# INSERT INTO fund(f_name,f_id,f_type,f_amount,risk_level,f_manager) VALUES ('国债',3,'货
币型',10000,'低',3);
INSERT 0 1
finance=# INSERT INTO fund(f_name,f_id,f_type,f_amount,risk_level,f_manager) VALUES ('沪深300指数'
,4,'指数型',10000,'中',4);
INSERT 0 1
```

查询插入结果如下：

一共 4 条数据。

```
finance=# select count(*) from fund;
 count
-------
     4
(1 row)
```

（6）对 property 表进行数据初始化

```
finance=# INSERT INTO property(pro_c_id,pro_id,pro_status,pro_quantity,pro_income,pro_purchase_tim
e) VALUES (5,1,'可用',4,8000,'2018-07-01');
INSERT 0 1
finance=# INSERT INTO property(pro_c_id,pro_id,pro_status,pro_quantity,pro_income,pro_purchase_tim
e) VALUES (10,2,'可用',4,8000,'2018-07-01');
INSERT 0 1
finance=# INSERT INTO property(pro_c_id,pro_id,pro_status,pro_quantity,pro_income,pro_purchase_tim
e) VALUES (15,3,'可用',4,8000,'2018-07-01');
INSERT 0 1
finance=# INSERT INTO property(pro_c_id,pro_id,pro_status,pro_quantity,pro_income,pro_purchase_tim
e) VALUES (20,4,'冻结',4,8000,'2018-07-01');
INSERT 0 1
```

查询插入结果如下：

一共 4 条数据。

```
finance=# select count(*) from property;
 count
-------
     4
(1 row)
```

5、手工插入一条数据

当 C 银行有新的信息需要加入数据库时，系统需要在对应的数据表中手动插入一条新的数据。因此，针对主键属性定义的场景，手动插入一条数据：

（1）在金融数据库的客户信息表中添加一个客户的信息。（属性冲突的场景）

c_id_card 和 c_phone 非唯一时，会出现如下报错：

```
finance=# INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (31,'李丽','l
ili@huawei.com','340211199301010005','18815650005','gaussdb_005');
ERROR:  duplicate key value violates unique constraint "client_c_id_card_key"
DETAIL:  Key (c_id_card)=(340211199301010005  ) already exists.
```

由于在表的创建过程中，实验定义了 c_id_card 和 c_phone 为唯一且非空（UNIQUE NOT NULL），所以当表中存在时，插入数据失败。

（2）在金融数据库的客户信息表中添加一个客户的信息。（插入成功的场景）

```
finance=# INSERT INTO client(c_id,c_name,c_mail,c_id_card,c_phone,c_password) VALUES (31,'李丽','l
ili@huawei.com','340211199301010031','18815650031','gaussdb_031');
INSERT 0 1
```

6、添加约束

在理财产品表、保险信息表和基金信息表中，都存在金额这个属性，在现实生活中，金额不会存在负数。因此针对表中金额的属性，增加大于 0 的约束条件：

（1）为 finances_product 表的 p_amount 列添加大于等于 0 的约束

```
finance=# ALTER table finances_product ADD CONSTRAINT c_p_mount CHECK (p_amount >=0);
ALTER TABLE
```

（2）尝试手工插入一条金额小于 0 的记录

```
finance=# INSERT INTO finances_product(p_name,p_id,p_description,p_amount,p_year) VALUES ('信贷资
产',10,'一般指银行作为委托人将通过发行理财产品募集资金委托给信托公司，信托公司作为受托人成立信托计
划，将信托资产购买理财产品发售银行或第三方信贷资产。',-10,6);
ERROR:  new row for relation "finances_product" violates check constraint "c_p_mount"
DETAIL:  Failing row contains (信贷资产, 10, 一般指银行作为委托人将通过发行理财产品募集..., -10, 6
).
```

执行失败，由于 p_amount 值为-10 ，为负数。说明约束添加成功。

（3）向 fund 表添加约束

为 fund 表的 f_amount 列添加大于等于 0 的约束。

```
finance=# ALTER table fund ADD CONSTRAINT c_f_mount CHECK (f_amount >=0);
ALTER TABLE
```

（4）向 insurance 表添加约束

为 insurance 表的 i_amount 列添加大于等于 0 的约束。

```
finance=# ALTER table insurance ADD CONSTRAINT c_i_mount CHECK (i_amount >=0);
ALTER TABLE
```

7、查询数据

（1）单表查询

查询银行卡信息表：

```
finance=# SELECT b_number,b_type FROM bank_card;
            b_number             |      b_type
---------------------------------+------------------
 6222021302020000001             | 信用卡
 6222021302020000002             | 信用卡
 6222021302020000003             | 信用卡
 6222021302020000004             | 信用卡
 6222021302020000005             | 信用卡
 6222021302020000006             | 信用卡
 6222021302020000007             | 信用卡
 6222021302020000008             | 信用卡
 6222021302020000009             | 信用卡
 6222021302020000010             | 信用卡
 6222021302020000011             | 储蓄卡
 6222021302020000012             | 储蓄卡
 6222021302020000013             | 储蓄卡
 6222021302020000014             | 储蓄卡
 6222021302020000015             | 储蓄卡
 6222021302020000016             | 储蓄卡
 6222021302020000017             | 储蓄卡
 6222021302020000018             | 储蓄卡
 6222021302020000019             | 储蓄卡
 6222021302020000020             | 储蓄卡
(20 rows)
```

（2）条件查询

查询资产信息中‘可用’的资产数据：

```
finance=# select * from property where pro_status='可用';
 pro_c_id | pro_id |     pro_status     | pro_quantity | pro_income |  pro_purchase_time
----------+--------+--------------------+--------------+------------+---------------------
        5 |      1 | 可用               |            4 |       8000 | 2018-07-01 00:00:00
       10 |      2 | 可用               |            4 |       8000 | 2018-07-01 00:00:00
       15 |      3 | 可用               |            4 |       8000 | 2018-07-01 00:00:00
(3 rows)
```

（3）聚合查询

查询用户表中有多少个用户：

```
finance=# SELECT count(*) FROM client;
 count
-------
    31
(1 row)
```

查询银行卡信息表中，储蓄卡和信用卡的个数：

```
finance=# SELECT b_type,COUNT(*) FROM bank_card GROUP BY b_type;
      b_type       | count
-------------------+-------
 储蓄卡            |    10
 信用卡            |    10
```

查询保险信息表中，保险金额的平均值：

```
finance=# SELECT AVG(i_amount) FROM insurance;
          avg
-----------------------
 2700.0000000000000000
(1 row)
```

查询保险信息表中保险金额的最大值和最小值所对应的险种和金额：

```
finance=# select i_name,i_amount from insurance where i_amount in (select max(i_amount) from
insurance)
finance-# union
finance-# select i_name,i_amount from insurance where i_amount in (select min(i_amount) from
insurance);
    i_name    | i_amount
--------------+----------
 意外保险     |     5000
 财产损失保险 |     1500
(2 rows)
```

（4）连接查询

① 半连接

查询用户编号在银行卡表中出现的用户的编号，用户姓名和身份证：

```
finance=# SELECT c_id,c_name,c_id_card FROM client WHERE EXISTS (SELECT * FROM bank_card WHER
E client.c_id = bank_card.b_c_id);
 c_id | c_name |      c_id_card
------+--------+---------------------
    1 | 张一   | 340211199301010001
    3 | 张三   | 340211199301010003
    5 | 张五   | 340211199301010005
    7 | 张七   | 340211199301010007
    9 | 张九   | 340211199301010009
   10 | 李一   | 340211199301010010
   12 | 李三   | 340211199301010012
   14 | 李五   | 340211199301010014
   16 | 李七   | 340211199301010016
   18 | 李九   | 340211199301010018
   19 | 王一   | 340211199301010019
   21 | 王三   | 340211199301010021
   23 | 王五   | 340211199301010023
   24 | 王六   | 340211199301010024
   26 | 王八   | 340211199301010026
   27 | 王九   | 340211199301010027
   29 | 钱二   | 340211199301010029
(17 rows)
```

半连接是一种特殊的连接类型，在 SQL 中没有指定的关键字，通过在 WHERE 后面使用 IN 或 EXISTS 子查询实现。当 IN/EXISTS 右侧的多行满足子查询的条件时，主查询也只返回一行与 EXISTS 子查询匹配的行，而不是复制左侧的行。

② 反连接

反连接是一种特殊的连接类型，在 SQL 中没有指定的关键字，通过在 WHERE 后面使用 NOT IN 或 NOT EXISTS 子查询实现。返回所有不满足条件的行。这个关系的概念跟半连接相反。

查询银行卡号不是‘6222021302020000001*’（*表示未知）的用户的编号，姓名和身份证：

```
finance=# SELECT c_id,c_name,c_id_card FROM client WHERE c_id NOT IN (SELECT b_c_id FROM bank_card WHE
RE b_number LIKE '622202130202000001_');
 c_id | c_name |      c_id_card
------+--------+---------------------
    1 | 张一   | 340211199301010001
    2 | 张二   | 340211199301010002
    3 | 张三   | 340211199301010003
    4 | 张四   | 340211199301010004
    5 | 张五   | 340211199301010005
    6 | 张六   | 340211199301010006
    7 | 张七   | 340211199301010007
    8 | 张八   | 340211199301010008
    9 | 张九   | 340211199301010009
   10 | 李一   | 340211199301010010
   11 | 李二   | 340211199301010011
   12 | 李三   | 340211199301010012
   13 | 李四   | 340211199301010013
   14 | 李五   | 340211199301010014
   15 | 李六   | 340211199301010015
   16 | 李七   | 340211199301010016
   17 | 李八   | 340211199301010017
   18 | 李九   | 340211199301010018
   19 | 王一   | 340211199301010019
   20 | 王二   | 340211199301010020
   21 | 王三   | 340211199301010021
   22 | 王四   | 340211199301010022
   23 | 王五   | 340211199301010023
   24 | 王六   | 340211199301010024
   25 | 王七   | 340211199301010025
   26 | 王八   | 340211199301010026
   27 | 王九   | 340211199301010027
   28 | 钱一   | 340211199301010028
   29 | 钱二   | 340211199301010029
   30 | 钱三   | 340211199301010030
   31 | 李丽   | 340211199301010031
(31 rows)
```

（5）子查询

通过子查询，查询保险产品中保险金额大于平均值的保险名称和适用人群：

```
finance=# SELECT i1.i_name,i1.i_amount,i1.i_person FROM insurance i1 WHERE i_amount > (SELECT avg(i_am
ount) FROM insurance i2);
  i_name  | i_amount |      i_person
----------+----------+--------------------
 人寿保险 |     3000 | 老人
 意外保险 |     5000 | 所有人
(2 rows)
```

（6）ORDER BY 和 GROUP BY

① ORDER BY 子句

按照保额降序查询保险编号大于 2 的保险名称，保额和适用人群：

```
finance=# SELECT i_name,i_amount,i_person FROM insurance WHERE i_id>2 ORDER BY i_amount DESC;
    i_name    | i_amount |     i_person
--------------+----------+------------------
 意外保险     |     5000 | 所有人
 医疗保险     |     2000 | 所有人
 财产损失保险 |     1500 | 中年人
(3 rows)
```

② GROUP BY 子句

查询各理财产品信息总数，按照 p_year 分组：

```
finance=# SELECT p_year,count(p_id) FROM finances_product GROUP BY p_year;
 p_year | count
--------+-------
      6 |     4
(1 row)
```

（7）HAVING 和 WITH AS

① HAVING 子句

查询保险金额，统计数量等于 2 的适用人群数：

```
finance=# SELECT i_person,count(i_amount) FROM insurance GROUP BY i_person HAVING count(i_amount)=2;
      i_person      | count
--------------------+-------
 老人               |     2
 所有人             |     2
```

HAVING 子句依附于 GROUP BY 子句而存在。

② WITH AS 子句

使用 WITH AS 查询基金信息表：

```
finance=# WITH temp AS (SELECT f_name,ln(f_amount) FROM fund ORDER BY f_manager DESC) SELECT * FROM te
mp;
   f_name    |        ln
-------------+------------------
 沪深300指数 | 9.21034037197618
 国债        | 9.21034037197618
 投资        | 9.21034037197618
 股票        | 9.21034037197618
(4 rows)
```

该语句为定义一个 SQL 片段，该 SQL 片段会被整个 SQL 语句用到。

可以使 SQL 语句的可读性更高。存储 SQL 片段的表与基本表不同，是一个虚表。数据库不存放对应的定义和数据，这些数据仍存放在原来的基本表中。若基本表中的数据发生变化，从存储 SQL 片段的表中查询出的数据也随之改变。

8、视图

视图是一个虚拟表，是 sql 的查询结果，其内容由查询定义。对于来自多张关联表的复杂查询，就不得不使用十分复杂的 SQL 语句进行查询，造成极差的体验感。使用视图之后，可以极大的简化操作，使用视图不需要关心相应表的结构、关联条件等。

（1）创建视图

针对“查询用户编号在银行卡表中出现的用户的编号，用户姓名和身份证” 的查询，创建视图：

```
finance=# CREATE VIEW v_client as SELECT c_id,c_name,c_id_card FROM client WHERE EXISTS (SELECT * FROM
 bank_card WHERE client.c_id = bank_card.b_c_id);
CREATE VIEW
```

使用视图进行查询：

```
finance=# SELECT * FROM v_client;
 c_id | c_name |      c_id_card
------+--------+---------------------
    1 | 张一   | 340211199301010001
    3 | 张三   | 340211199301010003
    5 | 张五   | 340211199301010005
    7 | 张七   | 340211199301010007
    9 | 张九   | 340211199301010009
   10 | 李一   | 340211199301010010
   12 | 李三   | 340211199301010012
   14 | 李五   | 340211199301010014
   16 | 李七   | 340211199301010016
   18 | 李九   | 340211199301010018
   19 | 王一   | 340211199301010019
   21 | 王三   | 340211199301010021
   23 | 王五   | 340211199301010023
   24 | 王六   | 340211199301010024
   26 | 王八   | 340211199301010026
   27 | 王九   | 340211199301010027
   29 | 钱二   | 340211199301010029
(17 rows)
```

（2）修改视图内容

修改视图，在原有查询的基础上，过滤出信用卡用户：

```
finance=# CREATE OR REPLACE VIEW v_client as SELECT c_id,c_name,c_id_card FROM client WHERE EXISTS (SE
LECT * FROM bank_card WHERE client.c_id = bank_card.b_c_id and bank_card.b_type='信用卡');
CREATE VIEW
```

使用视图查询：

```
finance=# SELECT * FROM v_client;
 c_id | c_name |      c_id_card
------+--------+---------------------
    1 | 张一   | 340211199301010001
    3 | 张三   | 340211199301010003
    5 | 张五   | 340211199301010005
    7 | 张七   | 340211199301010007
    9 | 张九   | 340211199301010009
   10 | 李一   | 340211199301010010
   12 | 李三   | 340211199301010012
   14 | 李五   | 340211199301010014
   16 | 李七   | 340211199301010016
   18 | 李九   | 340211199301010018
(10 rows)
```

（3）修改视图名

```
finance=# ALTER VIEW v_client RENAME TO v_client_new;
ALTER VIEW
```

（4）删除视图

将 v_client 视图删除，删除视图不影响基表：

```
finance=# DROP VIEW v_client_new;
DROP VIEW
```

9、索引

（1）创建索引

在普通表 property 上创建索引：

```
finance=# CREATE INDEX idx_property ON property(pro_c_id DESC,pro_income,pro_purchase_time);
CREATE INDEX
```

（2）重命名索引

在普通表 property 上重建及重命名索引：

```
finance=# DROP INDEX idx_property;
DROP INDEX
finance=# CREATE INDEX idx_property ON property(pro_c_id DESC,pro_income,pro_purchase_time);
CREATE INDEX
finance=# ALTER INDEX idx_property RENAME TO idx_property_temp;
ALTER INDEX
```

（3）删除索引

```
finance=# DROP INDEX idx_property_temp;
DROP INDEX
```

10、数据的修改和删除

（1）修改数据

修改/更新银行卡信息表中 b_c_id 小于 10 和客户信息表中 c_id 相同的记录的 b_type 字段。

查看表数据：

```
finance=# SELECT * FROM bank_card where b_c_id<10 ORDER BY b_c_id
finance-# ;
            b_number             |      b_type       | b_c_id
---------------------------------+-------------------+--------
 6222021302020000001             | 信用卡            |      1
 6222021302020000016             | 储蓄卡            |      3
 6222021302020000002             | 信用卡            |      3
 6222021302020000003             | 信用卡            |      5
 6222021302020000004             | 信用卡            |      7
 6222021302020000013             | 储蓄卡            |      7
 6222021302020000005             | 信用卡            |      9
(7 rows)
```

更新数据：

```
finance=# UPDATE bank_card SET bank_card.b_type='借记卡' from client where bank_card.b_c_id = client.c
_id and bank_card.b_c_id<10;
UPDATE 7
```

查询数据表：

```
finance=# SELECT * FROM bank_card ORDER BY b_c_id;
            b_number             |      b_type       | b_c_id
---------------------------------+-------------------+--------
 6222021302020000001             | 借记卡            |      1
 6222021302020000002             | 借记卡            |      3
 6222021302020000016             | 借记卡            |      3
 6222021302020000003             | 借记卡            |      5
 6222021302020000013             | 借记卡            |      7
 6222021302020000004             | 借记卡            |      7
 6222021302020000005             | 借记卡            |      9
 6222021302020000006             | 信用卡            |     10
 6222021302020000007             | 信用卡            |     12
 6222021302020000019             | 储蓄卡            |     12
 6222021302020000008             | 信用卡            |     14
 6222021302020000009             | 信用卡            |     16
 6222021302020000010             | 信用卡            |     18
 6222021302020000011             | 储蓄卡            |     19
 6222021302020000012             | 储蓄卡            |     21
 6222021302020000014             | 储蓄卡            |     23
 6222021302020000015             | 储蓄卡            |     24
 6222021302020000017             | 储蓄卡            |     26
 6222021302020000018             | 储蓄卡            |     27
 6222021302020000020             | 储蓄卡            |     29
(20 rows)
```

（2）删除指定数据：

删除基金信息表中编号小于 3 的行。

查询基金表：

```
finance=# SELECT * FROM fund;
   f_name    | f_id |      f_type       | f_amount |     risk_level      | f_manager
-------------+------+-------------------+----------+---------------------+-----------
 股票        |    1 | 股票型            |    10000 | 高                  |         1
 投资        |    2 | 债券型            |    10000 | 中                  |         2
 国债        |    3 | 货币型            |    10000 | 低                  |         3
 沪深300指数 |    4 | 指数型            |    10000 | 中                  |         4
(4 rows)
```

删除数据：

```
finance=# DELETE FROM fund WHERE f_id<3;
DELETE 2
```

查询删除结果：

```
finance=# SELECT * FROM fund;
   f_name    | f_id |      f_type       | f_amount |     risk_level      | f_manager
-------------+------+-------------------+----------+---------------------+-----------
 国债        |    3 | 货币型            |    10000 | 低                  |         3
 沪深300指数 |    4 | 指数型            |    10000 | 中                  |         4
(2 rows)
```

11、新用户的创建和授权

在本章中，假设 C 银行的某新员工想要在自己的用户下去访问 sys 用户下的金融数据库，则该员工需要向 sys 申请添加相关权限，具体操作如下：

（1）连接数据库后，进入 SQL 命令界面。创建用户 dbuser，密码为 Gauss#3demo。

```
finance=# CREATE USER dbuser IDENTIFIED BY 'Gauss#3demo';
CREATE ROLE
```

（2）给用户 dbuser 授予 finance 数据库下 bank_card 表的查询和插入权限，并将 SCHEMA 的权限也授予 dbuser 用户。

```
finance=# GRANT SELECT,INSERT ON finance.bank_card TO dbuser;
GRANT
finance=# GRANT ALL ON SCHEMA finance to dbuser;
GRANT
```

12、新用户连接数据库

（1）在 gsql 登录数据库，使用新用户连接。

使用操作系统 omm 用户在新的窗口登陆并执行以下命令，并输入对应的密码：

```
[omm@yyr-lab ~]$ gsql -d finance -U dbuser -p 26000 -r
Password for user dbuser:
gsql ((openGauss 1.1.0 build 392c0438) compiled at 2020-12-31 20:08:06 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

finance=>
```

（2）访问 finance 数据库的表 bank_card。

```
finance=> select * from finance. bank_card where b_c_id<10;
            b_number             |      b_type       | b_c_id
---------------------------------+-------------------+--------
 6222021302020000001             | 借记卡            |      1
 6222021302020000002             | 借记卡            |      3
 6222021302020000003             | 借记卡            |      5
 6222021302020000004             | 借记卡            |      7
 6222021302020000005             | 借记卡            |      9
 6222021302020000013             | 借记卡            |      7
 6222021302020000016             | 借记卡            |      3
(7 rows)
```

13、删除 schema

（1）使用管理员用户登陆 finance 数据库

使用操作系统 omm 用户使用 gsql，新建 session：

```
[omm@yyr-lab ~]$ gsql -d finance -p 26000;
gsql ((openGauss 1.1.0 build 392c0438) compiled at 2020-12-31 20:08:06 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

finance=#
```

（2）使用“\dn”查看数据库下的 schema

```
finance=# \dn
   List of schemas
    Name     | Owner
-------------+--------
 cstore      | omm
 dbe_perf    | omm
 dbuser      | dbuser
 finance     | omm
 pkg_service | omm
 public      | omm
 snapshot    | omm
(7 rows)
```

（3）设置默认查询为 finance

```
finance=# set search_path to finance;
SET
```

（4）使用“\dt”命令可以看到在 finance 中的对象

```
finance=# \dt
                        List of relations
 Schema  |       Name       | Type  | Owner |             Storage
---------+------------------+-------+-------+---------------------------------
 finance | bank_card        | table | omm   | {orientation=row,compression=no}
 finance | client           | table | omm   | {orientation=row,compression=no}
 finance | finances_product | table | omm   | {orientation=row,compression=no}
 finance | fund             | table | omm   | {orientation=row,compression=no}
 finance | insurance        | table | omm   | {orientation=row,compression=no}
 finance | property         | table | omm   | {orientation=row,compression=no}
(6 rows)
```

（5）使用 DROP SCHEMA 命令删除 finance 会有报错，因为 finance 下存在对象

使用 DROP SCHEMA…..CASCADE 删除，会将 finance 连同下的对象一起删除：

```
finance=# DROP SCHEMA finance CASCADE;
NOTICE:  drop cascades to 6 other objects
DETAIL:  drop cascades to table client
drop cascades to table bank_card
drop cascades to table finances_product
drop cascades to table insurance
drop cascades to table fund
drop cascades to table property
DROP SCHEMA
```

（6）使用“\dt”命令可以看到在 finance 和 public 中的对象，对象已删除

```
finance=# \dt
No relations found.
```

（二）使用 JDBC 连接数据库

1、准备连接环境

（1）修改数据库的 pg_hba.conf 文件

在 GS_HOME 中查找 pg_hba.conf 文件，本实验中数据库 GS_HOME 设置的为/gaussdb/data/db1，实际操作中 GS_HOME 地址可以查看安装时的配置文件：<PARAM name="dataNode1" value="/gaussdb/data/db1"/>。

```
[omm@yyr-lab ~]$ cd /gaussdb/data/db1
[omm@yyr-lab db1]$ vi pg_hba.conf
```

将以下内容添加进 pg_hba.conf 文件：

host all all 0.0.0.0/0 sha256

```
# "local" is for Unix domain socket connections only
local   all             all                                     trust
host all yyr 192.168.0.121/32 sha256
# IPv4 local connections:
host    all             all             127.0.0.1/32            trust
host    all    all    192.168.0.121/32    trust
host    all    all    0.0.0.0/0 sha256
# IPv6 local connections:
```

使用 omm 用户登陆，使用 gs_ctl 将策略生效：

```
[omm@yyr-lab db1]$ gs_ctl reload -D /gaussdb/data/db1/
[2025-12-29 22:31:13.745][4382][][gs_ctl]: gs_ctl reload ,datadir is /gaussdb/data/db1
server signaled
```

（2）使用 omm 用户登陆数据库给 dbuser 用户授权，并退出数据库

```
[omm@yyr-lab db1]$ gsql -d postgres -p 26000 -r
gsql ((openGauss 1.1.0 build 392c0438) compiled at 2020-12-31 20:08:06 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

postgres=# alter role dbuser createrole createdb;
ALTER ROLE
postgres=# \q
```

（3）修改数据库监听地址

在 GS_HOME 中，本实验中数据库 GS_HOME 设置的为/gaussdb/data/db1

```
[omm@yyr-lab db1]$ cd /gaussdb/data/db1
[omm@yyr-lab db1]$ vi postgresql.conf
```

将 listen_addresses 的值修改成为*

```
listen_addresses = '*'          # what IP address(es) to listen on;
                                # comma-separated list of addresses;
                                # defaults to 'localhost'; use '*' for all
                                # (change requires restart)
```

修改完成后重启数据库生效（-D 后面的数据库默认路径，需要根据实际情况进行修改）。

```
[2025-12-29 22:53:34.682][4491][][gs_ctl]:  done
[2025-12-29 22:53:34.682][4491][][gs_ctl]: server started (/gaussdb/data/db1)
```

（4）下载 Java 连接 openGauss 的驱动包，并将其导入对应的使用工具



（5）创建测试数据库 demo

使用 gsql 工具登陆数据库：

```
[omm@yyr-lab db1]$ gsql -d postgres -p 26000 -U dbuser -r
Password for user dbuser:
gsql ((openGauss 1.1.0 build 392c0438) compiled at 2020-12-31 20:08:06 commit 0 last mr  )
Non-SSL connection (SSL connection is recommended when requiring high-security)
Type "help" for help.

postgres=>
```

创建数据库 demo：

```
postgres=> create database demo ENCODING 'UTF8' template = template0;
CREATE DATABASE
postgres=> \connect demo;
Password for user dbuser:
Non-SSL connection (SSL connection is recommended when requiring high-security)
You are now connected to database "demo" as user "dbuser".
demo=>
```

（6）创建名为 demo 的 schema，并设置 demo 为当前的 schema

```
demo=> CREATE SCHEMA demo;
CREATE SCHEMA
```

将默认搜索路径设为 demo：

```
demo=> SET search_path TO demo;
SET
```

（7）创建测试表 websites

```
demo=> CREATE TABLE websites (
demo(>   id int NOT NULL,
demo(>   name char(20) NOT NULL DEFAULT '',
demo(>   url varchar(255) NOT NULL DEFAULT '',
demo(>   PRIMARY KEY (id)
demo(> );
NOTICE:  CREATE TABLE / PRIMARY KEY will create implicit index "websites_pkey" for table "websites"
CREATE TABLE
```

```
demo=> COMMENT ON COLUMN websites.name IS '站点名称';
COMMENT
```

（8）插入数据

```
demo=> INSERT INTO websites VALUES
demo-> ('1', 'openGauss', 'https://opengauss.org/zh/'),
demo-> ('2', '华为云', 'https://www.huaweicloud.com/'),
demo-> ('3', 'openEuler', 'https://openeuler.org/zh/'),
demo-> ('4', '华为support中心', 'https://support.huaweicloud.com/');
INSERT 0 4
```

2、确定 26000 端口是否放开

（1）打开华为云首页，登录后进入“控制台”，点击“弹性云服务器 ECS”进入 ECS 列表



（2）在云服务器控制台找到安装数据库主机的 ECS，点击查看基本信息，找到安全组



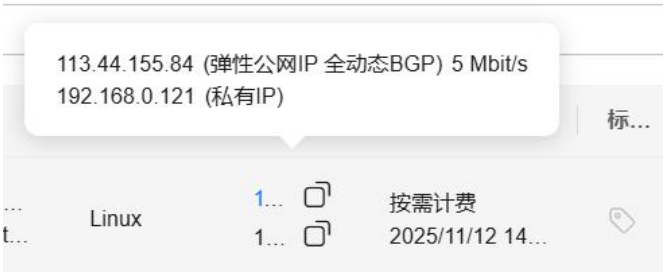
（3）点击进入安全组，选择“入方向规则”并“添加规则”，进行 26000 端口设置



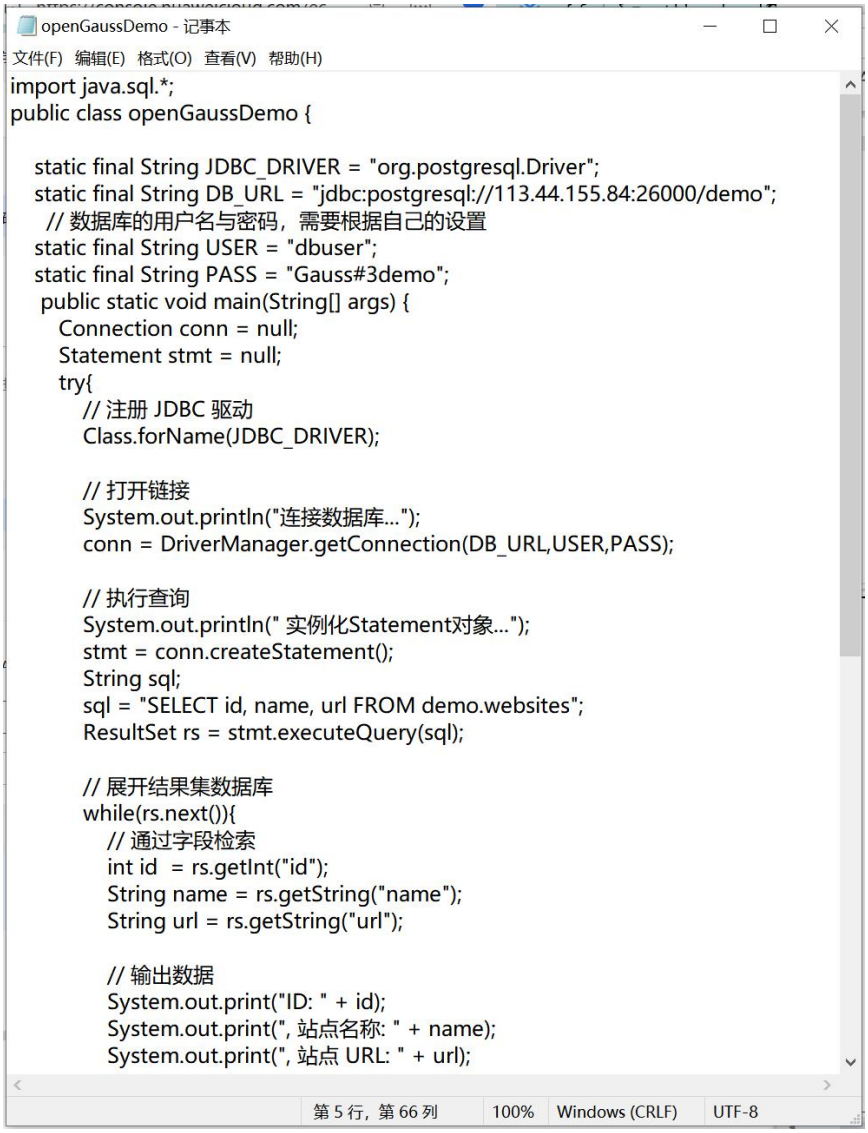
3、连接 openGauss 并执行 java 代码

（1）使用 Java 程序连接数据库并进行查询

根据实际情况替换红字内容，修改 jdbc:postgresql://公网 IP:26000/demo 中公网 IP 信息，USER = "dbuser"连接数据库的用户及密码 PASS = "Gauss#3demo"）。



在 d:\Download\中创建 openGaussDemo.java 文件，文件内容如下：



（2）在安装 Java 的本机，打开 cmd 对 Java 程序编译后执行

在 cmd 中，进入 d:\Download\目录，先对 Java 程序进行编译（进入 Java 程序的目录）。

```
D:\yyr\study\数据库系统>javac -encoding utf-8 -cp d:\yyr\study\数据库系统\postgresql.jar openGaussDemo.java
```

再执行以下命令：

```
D:\yyr\study\数据库系统> java -cp .;D:/yyr/study/数据库系统/postgresql.jar openGaussDemo
连接数据库...
 实例化Statement对象...
ID: 1, 站点名称: openGauss           , 站点 URL: https://opengauss.org/zh/
ID: 2, 站点名称: 华为云          , 站点 URL: https://www.huaweicloud.com/
ID: 3, 站点名称: openEuler           , 站点 URL: https://openeuler.org/zh/
ID: 4, 站点名称: 华为support中心 , 站点 URL: https://support.huaweicloud.com/
Goodbye!
```

三、心得总结（写出自己在完成实验过程中遇到的问题、解决方法，以及体会、收获等）（10%）

（一）出现的问题

1、在完成实验创建表的过程中，由于突然间的校园网断连，是的 CloudShell 的连接断开，需要重新连接服务器。此时服务器上已经创建好了数据库 finance，以及名为 finance 的 schema，需要重新以 omm 用户的身份连接到 finance 数据库，并将 finance 设置为当前的 schema。

由于断连之前使用的是 set 命令将默认搜索路径设为 finance，该命令的生效只仅限于当前会话，所以当会话断开重新连接后，需要重新设置搜索路径，来设置 schema。

切换到 omm 用户，连接数据库服务后，先连接到 finance 数据库：

```
postgres=# \connect finance
Non-SSL connection (SSL connection is recommended when requiring high-security)
You are now connected to database "finance" as user "omm".
```

通过 help 命令查询如何查看当前已有 schema：

```
finance=# help
You are using gsql, the command-line interface to gaussdb.
Type:  \copyright for distribution terms
       \h for help with SQL commands
       \? for help with gsql commands
       \g or terminate with semicolon to execute query
       \q to quit
```

```
\dn[S+] [PATTERN]      list schemas
```

```
finance=# \dn
   List of schemas
    Name     | Owner
-------------+-------
 cstore      | omm
 dbe_perf    | omm
 finance     | omm
 pkg_service | omm
 public      | omm
 snapshot    | omm
(6 rows)
```

重新设置搜索路径：

```
finance=# SET search_path TO finance;
SET
```

（二）收获与体会

本次实验通过以金融行业为场景，设计数据库模型，并使用华为 openGauss 构建金融场景下的数据库。从设计数据库的第一步开始设计 ER 图、建立合适的关系模式、通过对数据库中

的对象（表、约束、视图、索引等）创建，熟悉并掌握了 openGauss 数据库的相关操作，并通过对表中数据的增删改查，模拟金融场景下的业务实现，强化了建表、插入数据、修改更新数据、删除数据、查询等的学习。并结合金融场景下的新用户创建以及权限分配，模拟用户登录数据库。